



Assignment: 01

Design And Analysis of Algorithms

Mradul Sonkar | 23B0980
Instructor: Rohit Gurjar

Algorithm Overview:

The algorithm constructs a Minimum Spanning Tree using Kruskal's algorithm. It then tries to reduce the number of red edges while keeping the total weight of the tree under the threshold T . The key steps involved in the algorithm are as follows:

1. Formation of MST using Kruskal's Algorithm

- We sort edges in ascending order of weight. In case of a tie, we prefer the edge with blue color.
- I am using a combination of DisjointSet data structure and a vector of edges present in MST.

2. Optimizing MST by Swapping Edges

- We identify blue edges currently not in the MST, we recognize the cycles that would be formed on their addition.
- We then try to find the heaviest red edge present in the cycle and then swap it with the blue edge.
- We repeat until the weight of the MST becomes less than threshold ($\leq T$) or no more such swaps are possible, in case of which we output an error message.

3. Efficiency Enhancements

- LCA (Lowest Common Ancestor) + Binary Lifting is used to efficiently find the heaviest red edge in the cycle.

Time Complexity Analysis

- Constructing the Initial MST $\implies O(E \log E)$
(Complexity of sorting edges and using Kruskal's algorithm with Union-Find)
- Reducing MST Cost (Edge Replacement) $\implies O(E \log V)$
(Checking all edges outside the MST and trying to replace the heaviest edge in MST using LCA with binary lifting.)
- **Overall Complexity**

$$O(E \log E + E \log V)$$

Code Overview:

1. Structures Used:

(a) Edge

- To store information regarding an edge: start point (u), end point (v), weight (w), color (c).

(b) Graph

- To represent a graph, has number of vertices (V) and edges (E).
- Also a function to insert an edge in the graph and a list of all edges.

(c) Disjoint Set

- To track the connected components of the graph as we insert edges to form a MST.
- Has function to find the root of the connected component and to perform the union of two sets.

2. Functions Used:

(a) formMSTEdges

- This function just makes a MST for the given graph.
- We are using default comparator with an addition to break ties.
- If the weight of the edges is the same, we prefer blue edges.

(b) getResFromMST

- This function extracts the number of red edges and the cost of the MST, given the edge list of a MST.

(c) dfs

- Set up ancestor information, stores the direct parent of each node.
- Store the maximum weight edge on the path from the node to its parent.
- Calculate depth of each node.

(d) getMaxRedEdge

- This function finds the maximum weight red edge in the path between the nodes u and v in a tree using binary lifting.

(e) findAllBlueSwaps

- This function returns the list of all candidate blue edges that can be swapped with the red edges.

(f) doSwap

- This function swaps the red edge with the blue edge, and modify the total weight and count of red edges in the MST.

3. Additional Points:

- (a) I have used some macros to simplify my work. They are defined at the top of the code, and precisely are the following:

```
3
4  #define vi vector<int>
5  #define vE vector<Edge>
6  #define pii pair<int, int>
7
```

- (b) I have used fast IO optimization for taking inputs and outputting results faster.

```
268
269 // for faster IO
270 ios::sync_with_stdio(false);
271 cin.tie(NULL);
272 cout.tie(NULL);
273
```

4. Compilation Steps:

- (a) The code can be simply compiled using g++ compiler:

```
g++ 23B0980.cpp -o exec
```

and can be executed in the following way:

```
./exec
```

- (b) The executable asks for input from user and displays the result.

5. References:

- (a) Lecture Slides of course CS213 by Prof. Ashutosh Gupta
- (b) Codeforces Tutorial on Binary Lifting
- (c) LCA in a tree using Binary Lifting Technique - My implementation is inspired by the code available on this page.